МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №1

по дисциплине «Объектно-ориентированное программирование»

Студент гр. 5345 _____ Пантелеев А. С.

Преподаватель _____ Шестопалов Р. П.

Санкт-Петербург

2026

СОДЕРЖАНИЕ

Цель работы.....3
1 Описание архитектуры.....3
2 Описание классов.....6
2.1 Класс Point.....6
2.2 Класс Cell.....6
2.3 Класс Robot.....6
2.4 Класс Player.....7
2.5 Класс Factory.....7
2.6 Класс World.....7
2.7 Класс PathFinder.....7
2.8 Класс Game.....8
2.9 Класс Renderer.....8
2.10 Класс Input.....8
2.11 Класс Terminal.....8
3 Тестирование.....9
Заключение.....15

Цель работы

Цель работы — разработать пошаговую консольную игру на C++ и применить классы для хранения состояния роботов, игрового поля и управления ходами. Также нужно разделить игровую логику, ввод и вывод и показать связи классов на UML-диаграмме.

1 Описание архитектуры

В этой работе сделана пошаговая игра с роботами. Игрок управляет своим роботом на поле 30 × 30 клеток. На поле есть стены, участки болота, союзные и вражеские роботы и фабрика. Для сборки используется CMake, программа написана на C++20. Управление выполняется клавишами WASD, пропуск хода — R или пробел, выход — Q.

Исходный код разделён на каталоги game и terminal. В game находятся классы игрового мира, роботов, фабрики, поиска пути и отрисовки. В terminal находятся считывание команды и работа с терминалом. Заголовки лежат в include, реализации — в src. Такое разделение помогает не смешивать правила игры с чтением клавиатуры.

Главный объект программы — Game. В его полях находятся Terminal, Input, Renderer и World. Конструктор создаёт игрока, других роботов и фабрику, задаёт их начальные позиции и передаёт объекты миру. В методе Run сначала выводится поле, потом считывается команда игрока. Если действие выполнено, ходят компьютерные роботы и запускается обработка конца хода.

World хранит игровое поле как двумерный вектор Cell. Каждая клетка содержит тип местности, стоимость движения, признак видимости и тип находящегося на ней объекта: Empty, Robot или Factory. Координаты робота хранятся в Point. При перемещении World обновляет позицию робота и тип объекта в исходной и целевой клетках.

За время жизни игровых объектов отвечает World. Роботы, фабрики и игрок хранятся в std::unique_ptr. Клетка хранит только тип объекта, а нужный

экземпляр робота World находит по его координатам. Это разделяет владение объектами и учёт занятости поля.

Robot хранит здоровье, энергию, атаку, скорость, команду и позицию. Урон наносится при попытке войти в клетку вражеского робота. Player наследуется от Robot и дополнительно хранит опыт, уровень и радиус обзора. Когда игрок уничтожает врага, он получает опыт; при достижении порога увеличиваются характеристики. Встреча с союзником лечит игрока.

Скорость задаёт количество очков движения на один ход. Обычная клетка стоит одно очко, болото — два. На каждый шаг также тратится энергия. Если очков движения или энергии не хватает, робот остаётся на месте. В конце хода живые роботы восстанавливают часть энергии.

Компьютерные роботы используют PathFinder. Он строит карту минимальной стоимости пути к игроку алгоритмом Дейкстры: от клетки игрока последовательно рассматриваются четыре соседние клетки, а стоимость пути учитывает обычные клетки и болото. Затем робот выбирает соседнюю клетку, ведущую к цели с меньшими затратами.

Factory занимает несколько клеток и хранит образец робота. Через установленное число ходов она пытается создать нового врага слева от себя. Место появления должно находиться внутри поля и быть свободным. Созданный робот передаётся в World через std::unique_ptr.

Renderer получает мир только для чтения и собирает изображение поля в строку. Клетки, которые игрок ещё не видел, остаются скрытыми. В пределах текущего радиуса обзора отображаются местность, фабрики и роботы; за его пределами ранее открытые стены, болота и фабрики обозначаются знаком вопроса. Terminal выводит готовую строку и считывает клавиши без Enter. Input переводит символы клавиатуры в значения Command.

На рисунке 1 приведена UML-диаграмма классов программы. На ней показаны поля и методы классов, наследование Player от Robot, связи владения игровыми объектами и состояние клетки.

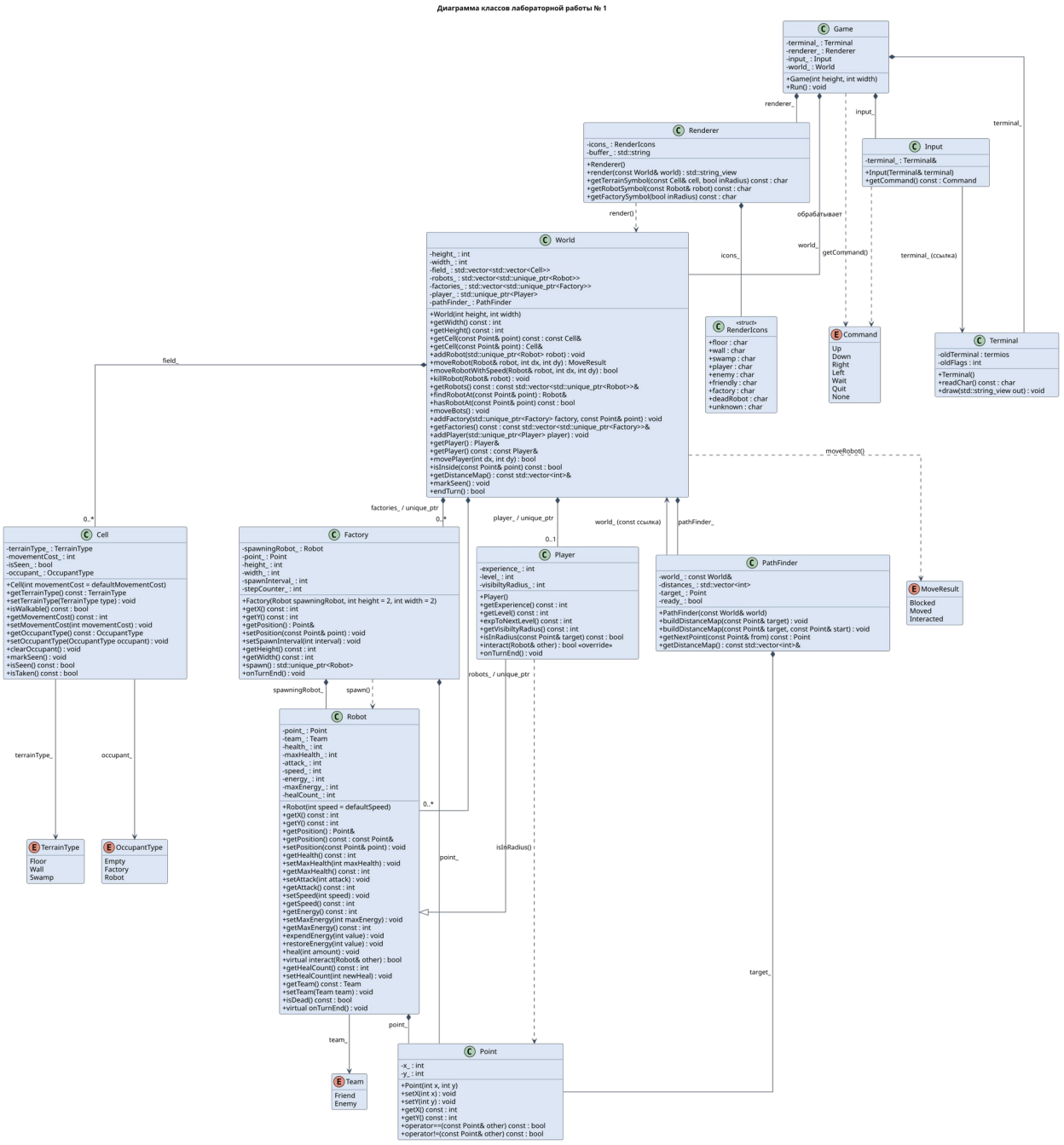


Рисунок 1 – UML-диаграмма классов программы

2 Описание классов

2.1 Класс Point

Point хранит координаты x и y. Конструктор задаёт их начальные значения, методы getX и getY возвращают координаты, а setX и setY позволяют их изменить. Операторы сравнения нужны, чтобы проверять, совпадают ли две клетки при поиске пути и движении.

2.2 Класс Cell

Cell описывает одну клетку поля. Она хранит тип местности, стоимость прохода, признак того, что игрок уже видел эту клетку, и значение OccupantType. Стена имеет нулевую стоимость прохода, обычная клетка — один, болото — два. Метод isWalkable учитывает стоимость прохода и наличие фабрики.

Значение OccupantType показывает, свободна клетка или занята роботом либо фабрикой. Методы setOccupantType и clearOccupant обновляют это состояние, а isTaken позволяет проверить занятость. Владение игровыми объектами остаётся у World.

2.3 Класс Robot

Robot — базовый класс для игровых роботов. В нём хранятся позиция, команда, текущее и максимальное здоровье, атака, скорость и энергия. Сеттеры не дают записать отрицательные значения здоровья и энергии или превысить установленный максимум.

Метод interact наносит урон роботу другой команды. Если команды совпадают, обычный Robot не атакует. onTurnEnd восстанавливает одну единицу энергии живому роботу. Деструктор виртуальный, поскольку Player наследуется от Robot.

2.4 Класс Player

Player наследует характеристики Robot и добавляет опыт, уровень и радиус видимости. Конструктор относит игрока к дружественной команде. Метод isInRadius проверяет, попадает ли выбранная позиция в круг обзора.

При взаимодействии с союзником игрок восстанавливает здоровье. За уничтожение противника добавляется опыт. После достижения порога уровень повышается, растут максимальное здоровье, атака и максимальная энергия. Эти правила записаны в методе gainExp.

2.5 Класс Factory

Factory хранит образец Robot, размер занимаемой области, позицию, интервал появления и счётчик ходов. В текущем уровне фабрика занимает квадрат 2 × 2 клетки. Метод spawn создаёт копию образца, когда наступает нужный ход, а onTurnEnd меняет счётчик. Проверку свободного места и добавление нового робота выполняет World.

2.6 Класс World

World объединяет поле и игровые объекты. Он создаёт стены и болота, хранит роботов и фабрики, а также единственного игрока. Методы addRobot, addFactory и addPlayer получают владение созданными объектами через std::unique_ptr. Метод findRobotAt находит робота по координатам, включая игрока.

При движении World проверяет границы поля, препятствия и занятость клетки. При встрече с другим роботом запускается взаимодействие. После перемещения World обновляет тип объекта в обеих клетках. Метод moveBots вызывает поиск пути для врагов. В конце хода восстанавливается энергия, обновляются фабрики и проверяется состояние игры.

2.7 Класс PathFinder

PathFinder хранит ссылку на World и рассчитывает расстояния до цели. Для этого используется алгоритм Дейкстры с очередью приоритетов. Карта строится от игрока в обратном направлении: для каждой клетки записывается

минимальная стоимость пути до него. Непроходимые и занятые другими роботами клетки не используются как обычные шаги маршрута.

Метод getNextPoint выбирает для робота соседнюю клетку, после перехода в которую путь к игроку будет дешевле. Стоимость входа в болото учитывается отдельно от обычной клетки.

2.8 Класс Game

Game задаёт начальные условия и запускает игровой цикл. В его состав входят World, Renderer, Input и Terminal. При создании мира добавляются игрок, союзники, враги и фабрика. Метод Run повторяет вывод поля, чтение команды и выполнение ходов, пока игрок не выйдет или игра не завершится.

2.9 Класс Renderer

Renderer превращает состояние World в текст для терминала. Сначала заполняется поле символами местности с учётом видимости, затем поверх рисуются фабрики, роботы и игрок. Внизу добавляются здоровье, энергия, уровень, опыт и подсказка по клавишам. Метод render возвращает представление готовой строки.

2.10 Класс Input

Input получает ссылку на Terminal и переводит нажатую клавишу в команду. WASD задают направление, R и пробел означают ожидание, Q завершает игру. Неизвестный символ возвращает Command::None, поэтому игровой ход не выполняется.

2.11 Класс Terminal

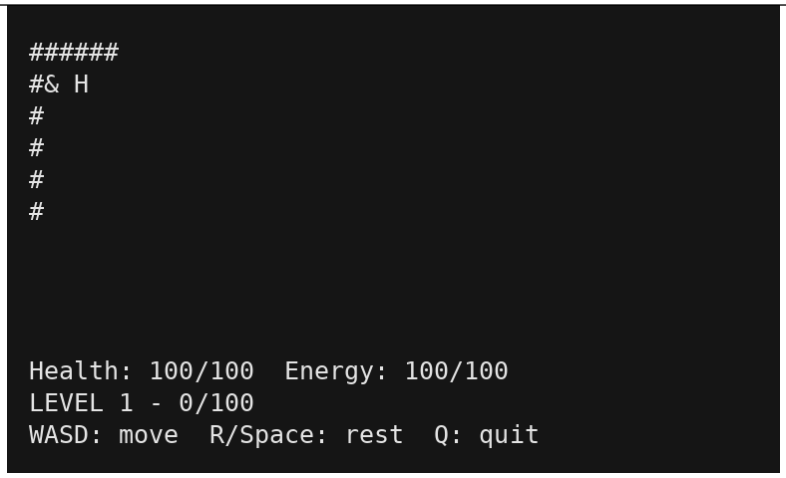
Terminal работает с вводом и выводом в консоли. В конструкторе он сохраняет прежние настройки и включает чтение по одному символу без отображения введённой клавиши. Метод draw выводит сформированную Renderer строку. При уничтожении объекта деструктор восстанавливает настройки терминала.

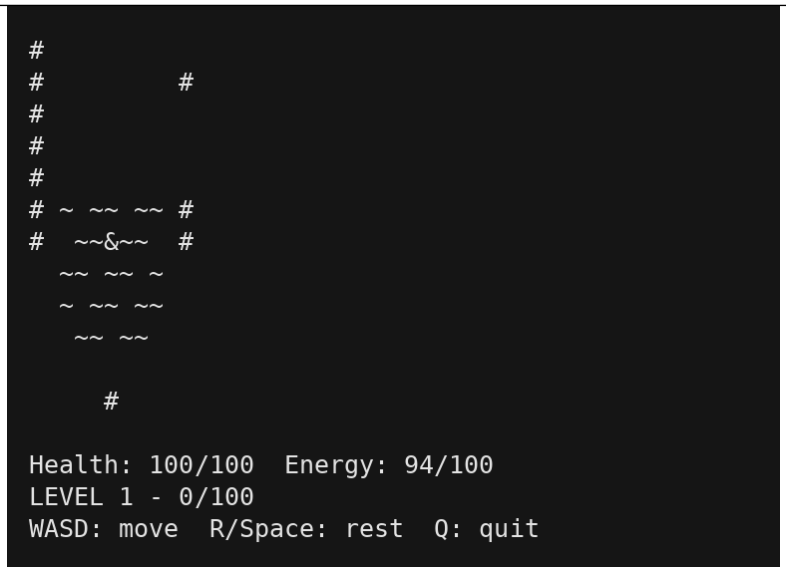
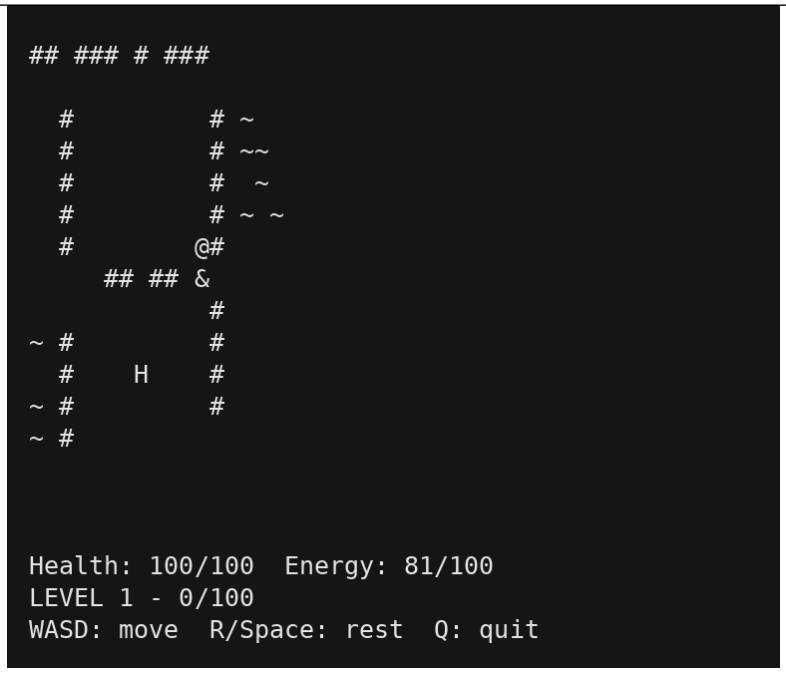
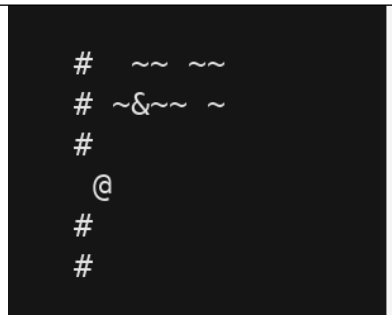
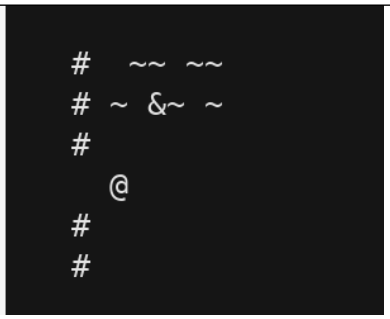
3 Тестирование

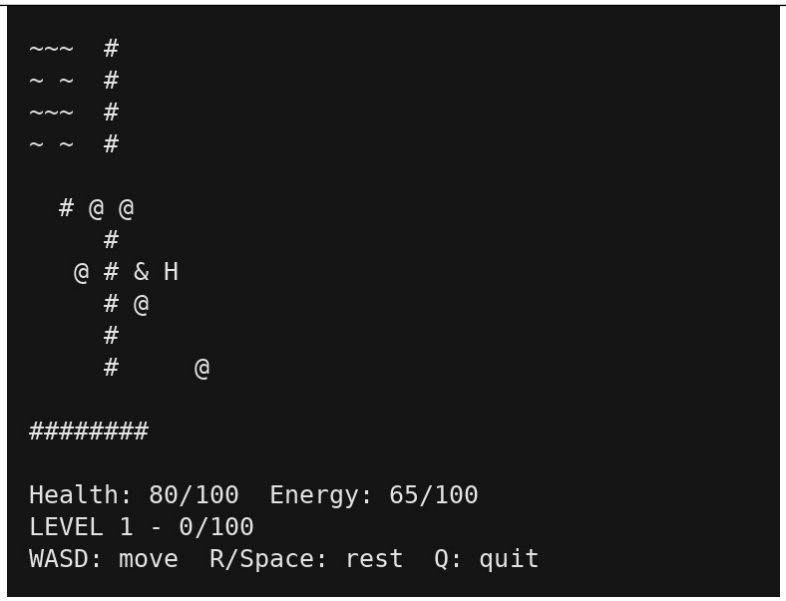
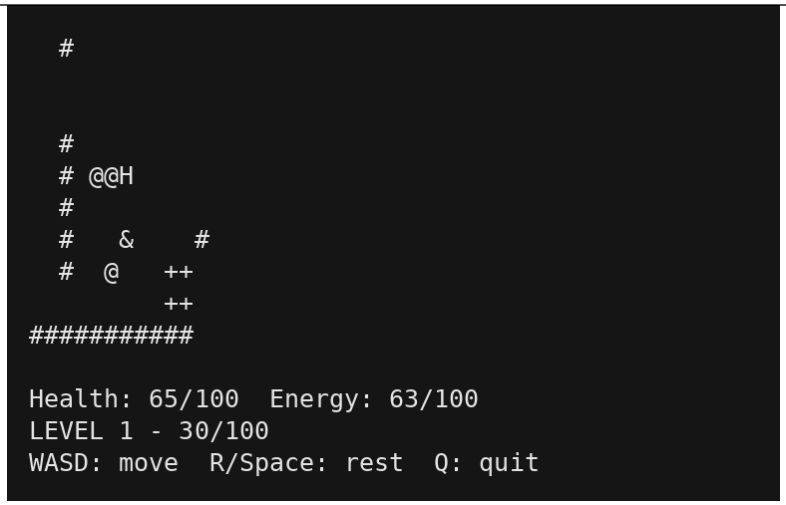
Проект собран через CMake с компилятором GCC и стандартом C++20. Я проверил игру в двух запусках: в первом прошёл северную и правую части поля, во втором двигался вдоль левого края, затем через южную часть к фабрике. По пути проверил перемещение, препятствия, болото, видимость, ходы противников, встречу с другими роботами и восстановление энергии. Фрагменты экрана приведены в таблице 1.

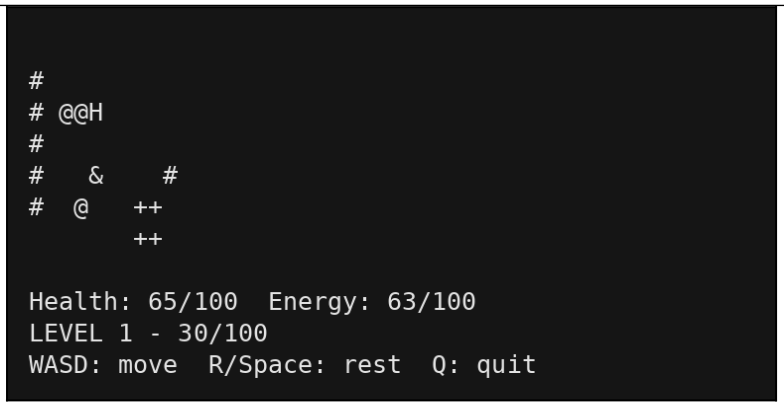
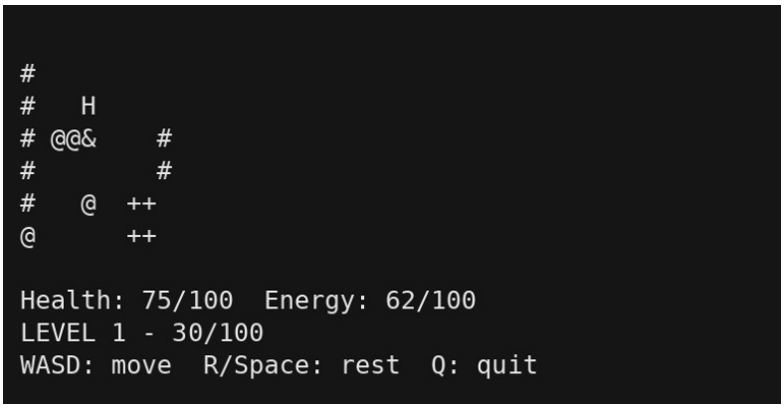
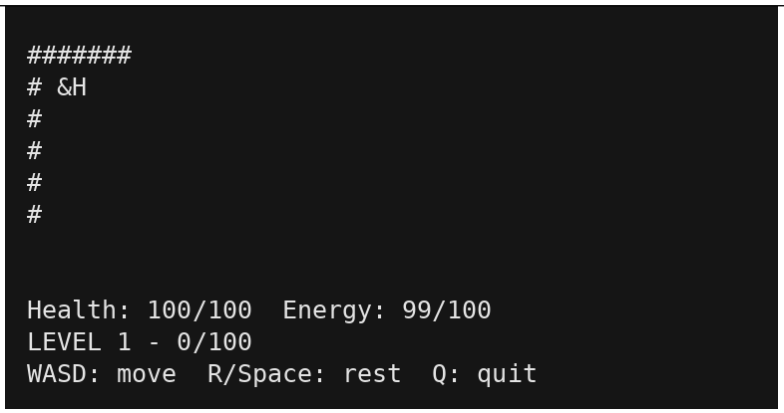
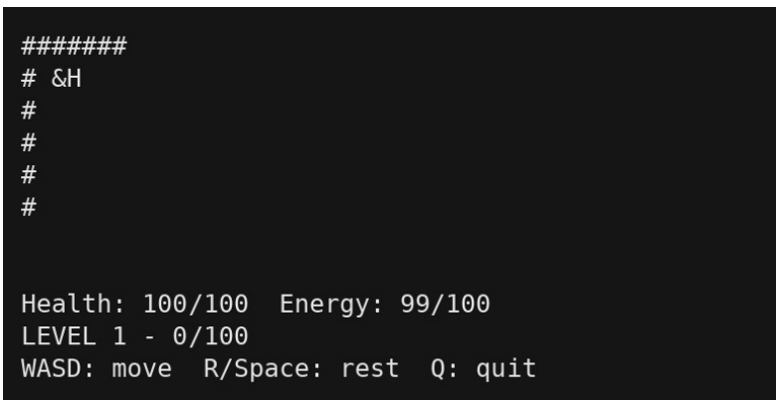
На поле видны союзники H, противники @ и фабрика +. После встречи с союзником здоровье выросло с 65 до 75. В отдельном сценарии с заданным начальным опытом я проверил бой и повышение уровня: после уничтожения врага уровень стал 2. Другой сценарий показывает появление врага рядом с фабрикой. Оба сценария используют те же классы игры, что и основной запуск.

Таблица 1 – результаты проверки программы

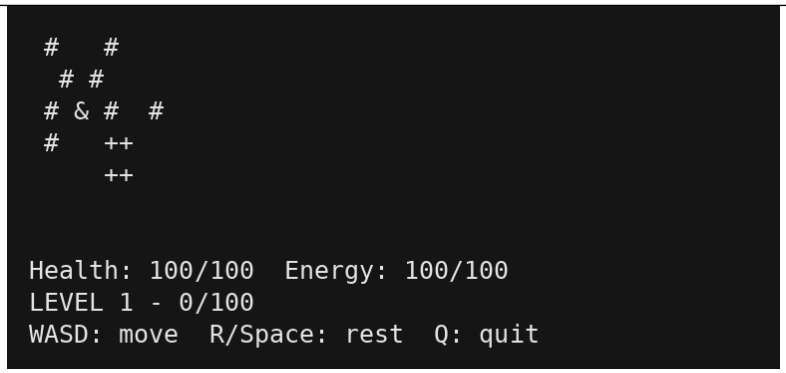
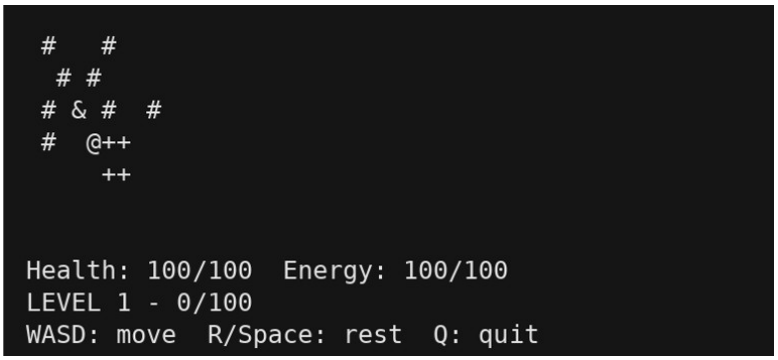
Входные данные	**Выходные данные**
Запуск игры	

Входные данные	Выходные данные
Болото и обзор	
Стены и роботы	
Ход бота	 

Входные данные	Выходные данные
Исследование поля	
Правый нижний угол: фабрика	

Входные данные	Выходные данные
Встреча с союзником	 
Попытка пройти в стену	 

Входные данные	Выходные данные
Десять ходов ожидания	``` ####### # &H # # # # Health: 100/100 Energy: 99/100 LEVEL 1 - 0/100 WASD: move R/Space: rest Q: quit ``` ``` ####### # &H # # # # Health: 100/100 Energy: 100/100 LEVEL 1 - 0/100 WASD: move R/Space: rest Q: quit ```
Бой и повышение уровня	``` ###### #&@ # # # # Health: 100/100 Energy: 100/100 LEVEL 1 - 90/100 WASD: move R/Space: rest Q: quit ``` ``` ###### #& # # # # Health: 100/110 Energy: 100/105 LEVEL 2 - 20/200 WASD: move R/Space: rest Q: quit ```

Входные данные	Выходные данные
Создание робота фабрикой	 

Заключение

В ходе лабораторной работы разработана пошаговая консольная игра с роботами. Для поля, роботов, игрока, фабрики, поиска пути, ввода и вывода созданы отдельные классы. Игрок может перемещаться, взаимодействовать с роботами и наблюдать открытую часть поля.

Основное состояние игры хранит World. Для владения динамически созданными объектами используются std::unique_ptr, а клетки отмечают тип находящегося на них объекта значением OccupantType. Компьютерные роботы ищут путь алгоритмом Дейкстры. Сборка, проход по разным частям карты и отдельные проверки боя, лечения, восстановления энергии и фабрики подтвердили работу основных возможностей игры.